Earners AI and ML by Aditya Jain sir - Lecture 18

HW

Problem 6 : Password Validator

```
Problem Statement
A password is valid if it contains

minimum 8 characters
at least one uppercase letter
at least one lowercase letter
at least one digit

Display

Strong Password

or

Weak Password
```

```
password = "Aditya789"

upper = False
lower = False
digit = False
n = len(password)
for c in password:
  if c.islower():
    lower = True
  elif c.isupper():
    upper = True
  elif c.isdigit():
    digit = True

if n >= 8 and upper==True and lower == True and digit == True:
  print("Strong Password")
else:
  print("Weak Password")
```

```
Strong Password
```

```
password = "Adityazzz"

upper = False
lower = False
digit = False

n = len(password)
for c in password:
  if c.islower():
    lower = True
  elif c.isupper():
    upper = True
  elif c.isdigit():
    digit = True

if n >= 8 and upper == True and lower == True and digit == True:
  print("Strong Password")
else:
  print("Weak Password")
```

```
Weak Password
```

```
#refined code

password = "Adityazzz"

upper = False
lower = False
digit = False

n = len(password)
for c in password:
  if c.islower():
    lower = True
  elif c.isupper():
    upper = True
  elif c.isdigit():
    digit = True

if n >= 8 and upper and lower and digit:
  print("Strong Password")
else:
  print("Weak Password")
```

```
Weak Password
```

```
#without system defined functions:

password = "Adityazzz"

upper = False
lower = False
digit = False

n = len(password)
for c in password:
  if c >= "a" and c <= "z":
    lower = True
  elif c >= "A" and c <= "Z":
    upper = True
  elif c >= "0" and c <= "9":
    digit = True

if n >= 8 and upper and lower and digit:
  print("Strong Password")
else:
  print("Weak Password")
```

```
Weak Password
```

Problem 7 : Cricket Scoreboard

```
Problem Statement

Take player names and runs for 5 players.

Store them in a dictionary.

Display

Highest scorer
Average score
```

```
scores = {
    "Virat" : 90,
    "Rohit" : 95,
    "Dhoni" : 100,
    "Hardik" : 50,
```

```
    "Bhumra" : 20
}
type(scores)
```

```
dict
```

```
(90+95+100+50+20)/5
```

```
71.0
```

```
mx_score = 0
total_score = 0
for key in scores:
  value = scores[key]

  if value > mx_score:
    mx_score = value
    mx_player = key

  total_score += value

print(mx_score)
print(mx_player)
print(total_score/len(scores))
```

```
100
Dhoni
71.0
```

```
#Bonus
# player = max(scores, key = scores.get)
# player
```

Problem 8 : Common Subjects

```
Problem Statement

Input two sets of subjects.

Display

Common subjects
Subjects only in first student
Subjects only in second student
```

```
s1 = {"Maths", "Science", "English"}
s2 = {"History", "Maths", "Science"}

common_subjects = s1 & s2
print(common_subjects)
```

```
{'Science', 'Maths'}
```

```
only_s1 = s1 - s2
print(only_s1)
```

```
{'English'}
```

```
only_s2 = s2 - s1
print(only_s2)
```

```
{'History'}
```

Problem 9 : Most Frequent Character

```
Problem Statement

Find the character that appears the maximum number of times in a string.
```

```
Ignore spaces.
```

```
s = "Python is the most amazing programming language for ML"

freq = {}

for c in s:
  if c != " ":
    if c not in freq:
      freq[c] = 1
    else:
      freq[c] += 1

print(freq)
```

```
{'P': 1, 'y': 1, 't': 3, 'h': 2, 'o': 4, 'n': 4, 'i': 3, 's': 2, 'e': 2, 'm': 4, 'a': 5, 'z': 1, 'g': 5, 'p': 1, 'r': 3, 'l': 1,
```

```
s = "Python is the most amazing programming language for ML today"

freq = {}

for c in s:
  if c == " ":
    continue

  if c not in freq:
    freq[c] = 1
  else:
    freq[c] += 1

print(freq)
```

```
{'P': 1, 'y': 2, 't': 4, 'h': 2, 'o': 5, 'n': 4, 'i': 3, 's': 2, 'e': 2, 'm': 4, 'a': 6, 'z': 1, 'g': 5, 'p': 1, 'r': 3, 'l': 1,
```

```
for k,v in freq.items():
  print(k,v)
```

```
P 1
y 2
t 4
h 2
o 5
n 4
i 3
s 2
e 2
m 4
a 6
z 1
g 5
p 1
r 3
l 1
u 1
f 1
M 1
L 1
d 1
```

```
mx_v = 0
for k,v in freq.items():
  if v > mx_v:
    mx_v = v
    mx_key = k

print(mx_key)
```

```
a
```

Object Oriented Programming (OOP) using Python

```
# Class --> Blueprint
# Object --> specific instance of that class

# eg:
# Class --> Car
# object --> Volvo, BMW, Suzuki

# object --> methods and its properties
```

```
class ClassName:
  x = 10
```

```
p1 = ClassName()
print(p1.x)
```

```
10
```

```
p2 = ClassName()
print(p2.x)

p3 = ClassName()
print(p3.x)
```

```
10
10
```

```
class Car:
  no_of_wheels = 4
```

```
c1 = Car()
print(c1.no_of_wheels)
```

```
4
```

```
c2 = Car()
print(c2.no_of_wheels)
```

```
4
```

```
class Person:
  name = "Raj"
  city = "Mumbai"
```

```
p1 = Person()
print(p1.name, p1.city)
```

```
Raj Mumbai
```

```
p2 = Person()
print(p2.name, p2.city)
```

```
Raj Mumbai
```

```
p3 = Person()
print(p3.name, p3.city)
```

```
Raj Mumbai
```

```
# __init__()

# __init__() --> is built-in method which is always executed when the class gets initiated (object is being getting created)
```

```
class Person:

  def __init__(self, name, city):
    self.name = name
    self.city = city
```

```
p1 = Person("Raj", "Mumbai")
```

```
p1 = Person("Raj", "Mumbai")
print(p1.name, p1.city)
```

```
Raj Mumbai
```

```
p2 = Person("Vivek", "Pune")
print(p2.name, p2.city)
```

```
Vivek Pune
```

```
p2 = Person("Riya", "Delhi")
print(p2.name, p2.city)
```

```
Riya Delhi
```

```
Start coding or generate with AI.
```